📄 **Traducción y Solucionario Oficial**

DOCKER — EJERCICIOS DE CREACIÓN DE IMÁGENES (SOLUCIONARIO)

- **Nota inicial:** En todas las soluciones, `<usuari>` es el nombre de usuario de Docker Hub del alumno. Cada comando se ejecuta dentro de la carpeta correspondiente (`~/docker-imatges/exN`), que contiene el `Dockerfile` y el `index.html`.

Ejercicio 1

Dockerfile:

Dockerfile

```
None
FROM nginx:1.27-alpine
RUN apk add --no-cache nano
COPY index.html /usr/share/nginx/html/index.html
```

Órdenes (Comandos):

Bash

```
None
docker build -t <usuari>/web-nginx:1.0 .
docker run -d --name web1 -p 8081:80 <usuari>/web-nginx:1.0
curl http://localhost:8081
docker exec web1 nano --version
```

Ejercicio 2

Dockerfile:

Dockerfile

```
None
FROM httpd:2.4.62
RUN apt-get update && \
    apt-get install -y curl && \
    rm -rf /var/lib/apt/lists/*
COPY index.html /usr/local/apache2/htdocs/index.html
```

Órdenes (Comandos):

Bash

```
None
docker build -t <usuari>/web-apache:1.0 .
docker run -d --name web2 -p 8082:80
<usuari>/web-apache:1.0
curl http://localhost:8082
docker exec web2 curl --version
```

Ejercicio 3

Dockerfile:

Dockerfile

```
None
FROM nginx:1.26
RUN apt-get update && \
    apt-get install -y nano && \
    rm -rf /var/lib/apt/lists/*
COPY index.html /usr/share/nginx/html/index.html
```

Órdenes (Comandos):

Bash

```
None
docker build -t <usuari>/web-nginx:2.0 .
docker run -d --name web3 -p 8083:80 <usuari>/web-nginx:2.0
curl http://localhost:8083
docker exec web3 nano --version
```

Ejercicio 4

Dockerfile:

Dockerfile

```
None
FROM httpd:2.4-alpine
RUN apk add --no-cache curl
```

```
COPY index.html /usr/local/apache2/htdocs/index.html
```

Órdenes (Comandos):

Bash

```
None
docker build -t <usuari>/web-apache:2.0 .
docker run -d --name web4 -p 8084:80
<usuari>/web-apache:2.0
curl http://localhost:8084
docker exec web4 curl --version
```

Notas importantes para la corrección:

- **Puerto interno:** El puerto interno siempre es el `80`, tanto en NGINX como en Apache. Un error típico es poner `-p 8081:8081`.
- **Alpine vs Debian:** En las imágenes Alpine se usa `apk add --no-cache` (sin actualización previa). En las de Debian hace falta `apt-get update` antes de la instalación y `-y` para que no se quede esperando confirmación interactiva.
- **Limpieza:** Limpiar `/var/lib/apt/lists` no es obligatorio, pero reduce el tamaño de la imagen (es buena práctica hacerlo dentro del mismo comando `RUN`).
- **Ruta de copia:** También es válido copiar a la carpeta sin especificar el nombre del archivo de destino (ej. `COPY index.html /usr/share/nginx/html/`).

Ejercicio 5 (Comparación de tamaños para el `README.md`)

Para comprobarlo ejecutamos:

Bash

```
None
docker images
```

- **Respuesta esperada:** Las imágenes más pequeñas son las basades en **Alpine** (`web-nginx:1.0` y `web-apache:2.0`), que ocupan unas pocas

decenas de MB. Las basadas en **Debian** (`web-apache:1.0` y `web-nginx:2.0`) ocupan alrededor de 150-200 MB (la medida exacta puede variar según la versión descargada).

- **Motivo:** Alpine es una distribución Linux mínima pensada para contenedores. Usa la librería `musl` en lugar de `glibc` y `BusyBox` en lugar de las herramientas GNU, conteniendo solo lo imprescindible. Debian incluye muchas más bibliotecas y utilidades, además de conservar las listas de paquetes si no se limpian.

Ejercicio 6 (Publicación en Docker Hub)

Bash

```
None
docker login
docker push <usuari>/web-nginx:1.0
docker push <usuari>/web-apache:1.0
docker push <usuari>/web-nginx:2.0
docker push <usuari>/web-apache:2.0
```

Nota: Si una imagen se creó sin el nombre de usuario, el `push` fallará con un error de acceso denegado (`denied: requested access to the resource is denied`). Se soluciona volviéndola a etiquetar, por ejemplo: `docker tag web-nginx:1.0 <usuari>/web-nginx:1.0`. La comprobación se hace en Docker Hub.

Ejercicio 7 (Entrega a GitHub)

Estructura del repositorio:

Plaintext

```
None
practica-dockerfile-1/
├── README.md
├── ex1/
│   ├── Dockerfile
│   └── index.html
├── ex2/  (ídem)
├── ex3/  (ídem)
└── ex4/  (ídem)
```

Órdenes para subirlo (desde la terminal):

Bash

```
None
cd ~/docker-imatges
git init
git add .
git commit -m "Pràctica Dockerfile 1"
git branch -M main
git remote add origin
https://github.com/<usuari>/practica-dockerfile-1.git
git push -u origin main
```

(Si tienes instalada la CLI de GitHub, puedes crearlo y subirlo directamente con un solo comando: `gh repo create practica-dockerfile-1 --public --source=. --push`*).* El archivo `README.md` debe incluir el nombre del alumno y la respuesta del ejercicio 5.